Dr.Sally - Observer Pattern Example

The Basic Weather Station system will have a WeatherData class which acts as the Subject, and a single Observer, and a Concrete Observer CurrentConditionsDisplay, which displays the current temperature and humidity.

1. Subject Interface

This interface allows attaching and notifying observers.

```java
import java.util.ArrayList;
import java.util.List;

public interface Subject {
    void registerObserver(Observer o); void removeObserver(Observer o);
    void notifyObservers();
}
```

1. WeatherData (Concrete Subject)

Implements the Subject interface and holds temperature and humidity data.

```java
public class WeatherData implements Subject {
      private List<Observer> observers observers = new ArrayList<>();
      private float
      temperature; private
      float humidity;

          public void
              registerObserver(Observer
              o) { observers.add(o);
      }
```

```
    public  void
        removeObserver(Observer
        o)  {
        observers.remove(o);
}

public  void  notifyObservers()  {
        for (Observer observer :
            observers) {
            observer.update(temperature,
            humidity);
    }}
```

```
    // Called to notify observers when measurements change
    public void measurementsChanged() {
        notifyObservers();
    }

    // Setter for temperature
    public void setTemperature(float temperature) {
        this.temperature = temperature;
        measurementsChanged(); // Notify observers of change
    }

    // Getter for temperature
    public float getTemperature() {
        return temperature;
    }

    // Setter for humidity
    public void setHumidity(float humidity) {
        this.humidity = humidity;
        measurementsChanged(); // Notify observers of change
    }

    // Getter for humidity
    public float getHumidity() {
        return humidity;
    }
}
```

2. Observer Interface

Defines an update method that observers must implement.

Java

```
public interface Observer {
```

```
    void  update(float  temperature, float  humidity);
}
```

3. CurrentConditionsDisplay (Concrete Observer)

Implements the Observer interface to display the current conditions.

```
Java
    public  class  CurrentConditionsDisplay  implements  Observer  {
    private  float  temperature;
    private  float  humidity;

@Override
    public  void  update(float  temperature,  float  humidity)  {
    this.temperature  =  temperature;
    this.humidity  =  humidity; display();
}

    public void display() {
    System.out.println("Current  conditions:  "  +  temperature  +  "F
    degrees  and  "  + humidity  +  "%  humidity");
    }
}
```

4. Weather Station (Main Application)

Demonstrates the interaction between the subject and observer.

```
Java
public void display() {
System.out.println("Current  conditions:  "  +  temperature  +  "F  degrees
and  "  + humidity  +  "%  humidity");
    }
}
weatherData.registerObserver(currentDisplay);
weatherData.registerObserver(currentDisplay);
      }
}
```